

BEEFTECH

Building, running and extending the BeefTech platform



BeefTech Mobile Data Collection Platform

Developer Manual

[Android developers](#)

[Backend developers](#)

[Maintainers](#)

[DevOps](#)



Contents

1. Introduction Everyone

What you are working on
The system at a glance

Repository map
How to read this manual

2. Getting set up Everyone

Prerequisites
First build
Running the backend

Running the app
Using a physical phone
Development accounts

3. Architecture Everyone

The big picture
Modules and dependencies
Layers inside a feature

App start-up
Navigation and roles

4. The local database Android

Room and SQLCipher
Where things live
The non-negotiable rules

Changing the schema
Testing a migration

5. Offline storage and sync Android · Backend

Life of a record
The pending_sync queue
Workers and the dispatcher
Batch names

What the server does with a batch
The 7-day policy
Device check-in

6. Authentication and security Android · Backend

Online sign-in
Offline sign-in
Sessions and tokens on the phone

Checking a request on the server
Record scope
Before production

7. The backend Backend

Code layout
Start-up
Configuration

Server schema changes
Anatomy of a route
Voiding, audit, email, reports

8. API reference Backend · Android

Conventions
Public
Field data

Reference data and sync policy
Administration

9. Testing and CI Everyone

Kinds of test
Android unit tests
Backend route tests

Continuous integration
Contributing



10. Recipes Everyone

Adding a new synced record type
Adding an endpoint

Adding a manager or admin screen
Adding a dependency

11. Building, releasing and deploying Maintainers

Building the APK
Deploying the backend

Rebuilding the manuals

12. Troubleshooting and reference Everyone

Troubleshooting
Known gaps
Toolchain and versions

Other documents
Glossary



CHAPTER 1

Introduction

What you are working on

BeefTech is an offline-first Android app for feedlot management, livestock traceability and farm data capture, plus a Kotlin/Ktor server that the phones sync into. Field teams often work for days without signal, so the phone is the system of record until the server has confirmed each record.

This manual is for the people who build and maintain the platform. It explains how the code is organised, how data moves from a phone to the server, the rules that keep field data safe, and step-by-step recipes for the changes you will make most often. The [User Manual](#) describes the same app from the point of view of the people who use it; read its Chapter 3 (working offline and syncing) for the user-facing view of what this manual describes in code.

ANDROID**App developers**

Feature modules, Compose screens, the encrypted Room database and the sync workers. Chapters 3–6, 9, 10.

BACKEND**Server developers**

The Ktor API, its SQLite schema, auth, scoping, email and reports. Chapters 5–10.

MAINTAINERS**Release & DevOps**

Builds, CI, the Render deployment and the manuals themselves. Chapters 2, 9, 11, 12.

The system at a glance

Part	What it is
Android app (<code>:demoapp</code>)	The one installable app. It wires eight Android library modules together behind a PIN login, a bottom navigation bar (Home, Calves, Traceability, Feed, More) and the Day-7 sync policy.
Feature modules (<code>:android:*</code>)	Calf registration, farmer registration, farm traceability, feed crib, management (manager/admin screens), tag scanner and authentication. Each owns its screens, ViewModels, API client and sync worker.
Local database (<code>:android:database</code>)	Room over SQLCipher-encrypted SQLite. Every entity, DAO, migration and the sync bookkeeping lives here. Schema version 47.
Backend (<code>:backend:api</code>)	Ktor 3 on Netty, Exposed over SQLite. Accepts batch uploads, upserts by record GUID, scopes reads by role, sends notification emails, produces reports and PDFs.
Hosting	The backend is built by the <code>Dockerfile</code> and deployed to Render from <code>render.yaml</code> .

Repository map

Everything is one Gradle build. Always build from the repository root with the wrapper.



```

beeftech-mobile-data-platform/
├── android/
│   ├── authentication/      # PIN login, AuthGate, SessionStore, AuthApiClient
│   ├── calf-registration/   # 4-step calf capture, photos, sync client + worker
│   ├── database/           # Room + SQLCipher: entities, DAOs, migrations, security,
│   └── BackendConfig
│       └── farm-traceability/ # movements, treatments, mortalities, costs, purchases,
│           assignments
│               ├── farmer-registration/ # farmer onboarding Activities, farmer list, sync worker
│               ├── feed-crib/          # crib readings: home, crib detail, today's session, sync worker
│               ├── management/         # online-only manager/admin screens over ManagementApiClient
│               └── tag-scanner/         # CameraX + ML Kit ear-tag scanning and colour detection
├── backend/
│   ├── api/                 # the Ktor server
│   ├── authentication/      # empty placeholder
│   └── sync/                 # empty placeholder
├── demoapp/                 # the installable Android app
├── docs/                    # database notes, plans, user + developer manuals
├── gradle/libs.versions.toml # version catalog
├── .github/workflows/ci.yml  # unit tests, lint, database migration tests
├── Dockerfile · render.yaml # backend deployment
└── settings.gradle.kts      # module list and repositories

```

IGNORE

`android/` still holds a leftover standalone Gradle setup (`android/gradlew` , `android/gradle/wrapper/` , `android/settings.gradle.kts.backup` , `android/gradle.properties`) from before the modules were merged into the root build. Never build from inside `android/` .

How to read this manual

- `monospace` marks code: class names, file paths, Gradle tasks, endpoints, columns.
- Paths are relative to the repository root. Kotlin packages are shortened where obvious, e.g. `feedcrib/data/FeedCribRepository.kt` inside `android/feed-crib/src/main/java/com/beeftech/` .
- Dark blocks are commands or code you can copy. Commands run from the repository root.

NOTE

Short notes add useful background.

TIP

Tips make a task faster or easier.

IMPORTANT

Things that will break a build, a sync or a test if ignored.

**DANGER**

Mistakes that can lose field data or weaken security.

The code is the source of truth. Where this manual and the code disagree, trust the code and fix the manual (see [Rebuilding the manuals](#)).



CHAPTER 2

Getting set up

Prerequisites

Requirement	Version	Notes
JDK	17	The Gradle daemon is pinned to Java 17 in <code>gradle/gradle-daemon-jvm.properties</code> . A newer JDK on <code>PATH</code> is fine as long as a JDK 17 is installed; otherwise Gradle tries to download one and fails on restricted networks.
Android SDK	API 35, build-tools for AGP 8.13.2	Modules use <code>compileSdk</code> 34 or 35 and <code>minSdk</code> 24 (23 for <code>:android:database</code>).
Android Studio	Ladybug or newer	Optional if you only use the command line.
Gradle	Wrapper (8.13)	Always <code>./gradlew</code> , never a system <code>gradle</code> . On Windows use <code>gradlew.bat</code> .

First build

- 1 Clone the repository and open a terminal in its root.
- 2 Tell Gradle where the Android SDK is. `local.properties` is git-ignored, so every machine needs its own:

```
echo "sdk.dir=$HOME/Android/Sdk" > local.properties # macOS: $HOME/Library/Android/sdk
```

- 3 Check the toolchain. *Daemon JVM: Compatible with Java 17* means the pin was satisfied:

```
./gradlew --version
```

- 4 Run the same fast checks CI runs:

```
./gradlew testDebugUnitTest test
```

Running the backend

```
./gradlew :backend:api:run -Dbeeftech.seed.dev=true  
curl http://localhost:8081/ # BeefTech Backend API is running
```

- Listens on `0.0.0.0:8081` (or `$PORT`).
- Creates its SQLite database at `./data/beeftech-backend.db`, relative to the working directory, and keeps calf photos in a `media/` folder next to it.
- Seeds reference data (diseases, treatment types, cost types) and the feed crib reading codes on every start. Values an admin deactivated stay deactivated.



- `-Dbeeftech.seed.dev=true` also seeds a development site, three users and feed cribs `A01` – `A08` (see [Development accounts](#)).

To use a throw-away database, pass another JDBC URL. Any `beeftech.*` `-D` flag is forwarded to the `run` task:

```
./gradlew :backend:api:run -Dbeeftech.seed.dev=true -Dbeeftech.db.url="jdbc:sqlite:/tmp/beeftech.db"
```

Smoke-test a sign-in. `device_id` is required, and the first successful sign-in binds the user to that device:

```
curl -X POST http://localhost:8081/api/auth/login \
-H 'Content-Type: application/json' \
-d '{"username":"jvdm","pin":"30003","device_id":"TEST_DEV"}'
```

Running the app

Start an emulator, then in a second terminal:

```
./gradlew :demoapp:installDebug
adb shell am start -n com.beeftech.demoapp/.MainActivity
```

Or open the repository root in Android Studio and run the **demoapp** configuration. A debug build talks to `http://10.0.2.2:8081/`, which is how the emulator sees the backend running on your machine.

Build type	Backend base URL (<code>BuildConfig.BACKEND_BASE_URL</code>)
debug	<code>http://10.0.2.2:8081/</code> , unless overridden with <code>-Pbeeftech.baseUrl</code>
release	<code>https://beeftech-backend.onrender.com/</code>

Both are set in `demoapp/build.gradle.kts`. At start-up `BeefTechApplication` copies the value into `BackendConfig`, and every API client defaults its `baseUrl` to `BackendConfig.baseUrl`.

Using a physical phone

`10.0.2.2` only exists inside the emulator. On a real phone, point a debug build at your workstation's LAN address (both on the same network), or at the hosted backend:

```
./gradlew :demoapp:installDebug -Pbeeftech.baseUrl=http://192.168.1.20:8081/
./gradlew :demoapp:installDebug -Pbeeftech.baseUrl=https://beeftech-backend.onrender.com/
```

NOTE

Cleartext HTTP to a local backend works because `android:usesCleartextTraffic="true"` is set in the `demoapp` and `calf-registration` manifests. That also applies to release builds; see [Known gaps](#).



Development accounts

With the dev seed enabled, these users exist on site `dev-site-1` "Dev Feedlot" (farm code `S001`):

Username	PIN	Role	Site	Good for testing
<code>admin</code>	<code>10001</code>	1 — Administrator	none (sees all)	Admin area, cross-site reads
<code>fmanager</code>	<code>20002</code>	2 — Manager	<code>dev-site-1</code>	Dashboard, Reports, Records, Team, Feed
<code>jvdm</code>	<code>30003</code>	3 — Worker	<code>dev-site-1</code>	Field capture, offline sign-in, sync

TIP

The Feed tab needs a user with a site. `admin` has none, so `GET /api/feed-cribs` returns 400 for it unless the request names a site with `?siteId=`.

DEVICE BINDING

Each user is bound to the first device that signs in. Signing in as the same user from a second emulator returns `409`. Unbind from More → Team as a manager, call `POST /api/users/{id}/unbind-device`, or delete your local backend database.



CHAPTER 3

Architecture

The big picture

Every record is written to the encrypted local database first and queued. Sync is a separate, retryable step that never deletes local data because a request failed.

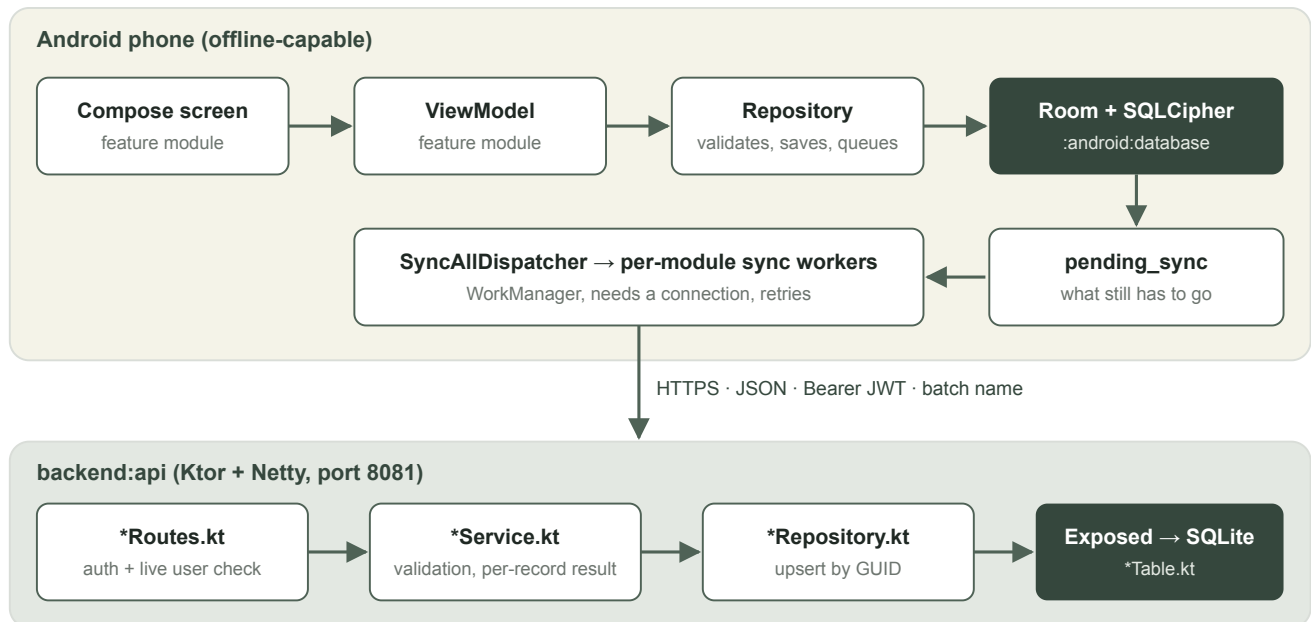


Figure 3.1 — From a screen on the phone to a row on the server

1. **Capture.** Compose screens validate at the point of entry.
2. **Encrypted local write.** The repository writes the record to the SQLCipher database with its audit metadata (record GUID, capture time, device ID, GPS where available, user ID).
3. **Queue.** A row in `pending_sync` (or the record's own `sync_status`) says it still has to be sent.
4. **Sync.** A WorkManager worker posts the waiting records in a batch with the session's JWT.
5. **Acknowledge.** The server upserts each record by GUID and stamps the submitter and site from the token. The phone marks the record `SYNCED` and clears its queue row.
6. **Check-in.** Separately, `DeviceCheckInWorker` downloads reference data and the sync policy and uploads the phone's security events.

Modules and dependencies

Only `:demoapp` is an Android application. Every feature module depends on `:android:database`; nothing depends on `:demoapp`.



Module	Package	Depends on	Holds
<code>:demoapp</code>	<code>com.beeftech.demoapp</code>	all features	Application, MainActivity, navigation chrome, <code>SyncAllDispatcher</code> , scheduled sync, check-in, sync-policy gate and guard
<code>:android:database</code>	<code>com.beeftech.database</code>	—	Room database, 45 entities, DAOs, migrations, shared repositories, security, <code>BackendConfig</code> , naming utilities
<code>:android:authentication</code>	<code>...authentication</code>	database	Online and offline PIN login, <code>AuthGate</code> , <code>SessionStore</code> / <code>EncryptedSessionStore</code>
<code>:android:calf-registration</code>	<code>...calfregistration</code>	database, tag-scanner	4-step calf capture, registered list, photos, <code>CalfRegistrationSyncWorker</code>
<code>:android:farm-traceability</code>	<code>...farmtraceability</code>	database, tag-scanner	Animal record, movements, purchases, location & feed, treatments, costs, mortalities, farmer–animal assignments and their workers
<code>:android:farmer-registration</code>	<code>...farmerregistration</code>	database	Farmer onboarding Activities, farmer list with sync pills, <code>FarmerSyncWorker</code>
<code>:android:feed-crib</code>	<code>...feedcrib</code>	database	Feed tab: crib search, 3×3 readings grid, today's session, <code>FeedCribSyncWorker</code>
<code>:android:management</code>	<code>...management</code>	database	Online-only Dashboard, Reports, Records, Team and Admin screens; reference data, sync policy and security event pulls
<code>:android:tag-scanner</code>	<code>...tagscanner</code>	database	CameraX + ML Kit ear-tag scanner, tag text parser, tag colour classifier
<code>:backend:api</code>	<code>com.beeftech.backend.api</code>	—	The Ktor server

NOTE

`:backend:authentication` and `:backend:sync` are empty placeholders declared in `settings.gradle.kts`. They configure but build nothing.



Layers inside a feature

Feature modules follow the same shape. `:android:feed-crib` is the newest and cleanest example; use it as your template.

```
android/feed-crib/src/main/java/com/beeftech/feedcrib/
├── data/
│   ├── FeedCribApiClient.kt      # Ktor client: one method per endpoint, returns Result<T>
│   ├── FeedCribNetworkModels.kt # @Serializable request/response DTOs
│   ├── FeedCribMappers.kt       # DTO ≠ Room entity
│   ├── FeedCribRepository.kt    # save locally, queue, try to send, reconcile results
│   ├── FeedCribCaptureContext.kt # device ID, capture time, GPS for a new record
│   └── SlotAllocator.kt         # pure domain logic, unit-tested on its own
├── ui/                          # Compose screens and components
├── viewmodel/                   # ViewModel + factory
└── worker/FeedCribSyncWorker.kt # CoroutineWorker: retries until nothing is waiting
```

- **UI** only talks to its ViewModel. It collects `StateFlow` / `Flow` and sends user intents.
- **ViewModels** talk to repositories, never to DAOs or HTTP clients.
- **Repositories** own the business rules: validation, the local write, queuing, calling the API client and applying per-record results. Their collaborators (DAO, API client, clock, user ID, time zone) are constructor parameters so tests can pass fakes.
- **Entities and DAOs** always live in `:android:database`, even when only one feature uses them, because there is one database and one migration chain.
- **API clients** take a `TokenProvider`, default their `baseUrl` to `BackendConfig.baseUrl`, and report a 401 with `tokenProvider.reportUnauthorized(...)`.

App start-up

`BeefTechApplication.onCreate` runs before anything else, including when `WorkManager` starts the process in the background for a scheduled sync:

- 1 Configures `BackendConfig` from `BuildConfig.BACKEND_BASE_URL`. This must happen before any API client is built.
- 2 Creates `EncryptedSessionStore` and registers it with `TokenProviderRegistry` and `CurrentUserIdRegistry`, so background workers can get a token and the current user without an Activity.
- 3 Registers `SyncPolicyActivityGuard`, which enforces the Day-7 lock in every Activity, including the former registration Activities started outside `MainActivity`.
- 4 Calls `ScheduledSyncScheduler.schedule`: two daily periodic jobs at 05:00 and 18:00 that run `ScheduledBatchSyncWorker`. It also cancels the legacy 15-minute periodic jobs.
- 5 Starts `SunlightBrightnessController`, which raises screen brightness in strong sunlight.

`MainActivity` then opens the encrypted database on a background thread through `DatabaseProvider.initialize(context, passphrase)`, seeds one demo calf (tag `BLu00000001`) if it does not exist, and shows `PolicyAwareAuthGate`: the PIN login (`AuthGate`), a device check-in, then `SyncPolicyGate`, which re-evaluates the unsynced-data policy every 60 seconds.

**DEMO PASSPHRASE**

`MainActivity` opens SQLCipher with the hard-coded passphrase `beeftech-demo-passphrase`. It must be replaced with the Android KeyStore-backed key (`DatabaseKeyProvider` / `SecureDatabaseInitializer` in `database/security/`) before production. Changing it on an existing install needs a re-key step, or the phone can no longer open its own data.

Navigation and roles

`demoapp/AppTab.kt` defines the destinations. Every role gets the same five bottom tabs; the More menu grows with the role:

Role (id)	Bottom tabs	Extra More entries
Worker (3)	Home, Calves, Traceability, Feed, More	— (My activity, Log out only)
Manager (2)	same	Dashboard, Reports, Records, Team
Admin (1)	same	Dashboard, Reports, Records, Team, Admin

Hiding a screen in the app is a convenience, not security. The server checks the role on every request ([Chapter 6](#)).



CHAPTER 4

The local database

Room and SQLCipher

All persistent app data lives in one Room database, `BeefTechDatabase`, stored in a SQLCipher-encrypted SQLite file. Schema version **47**; `BeefTechDatabase.VERSION` is the source of truth.

- `DatabaseProvider.initialize(context, passphrase: ByteArray)` is the single entry point. It returns a `DatabaseResult` and keeps one open instance for the process.
- Workers call `DatabaseProvider.getDatabase()`. It is `null` until `MainActivity` has unlocked the database, so every worker returns `Result.retry()` in that case instead of failing.
- `DatabaseFactory.create` builds the database with SQLCipher's `SupportOpenHelperFactory` and registers every migration.
- Schemas are exported (`exportSchema = true`) to `android/database/schemas/com.beeftech.database.BeefTechDatabase/` and committed.

Where things live

Path in	Contents
<code>android/database/.../database/</code>	
<code>BeefTechDatabase.kt</code>	The <code>@Database</code> class, its entity list, DAO accessors, <code>VERSION</code> and migrations <code>9→10</code> to <code>46→47</code> in the companion object
<code>DatabaseFactory.kt</code>	Migrations <code>1→2</code> to <code>8→9</code> , the <code>guarded(...)</code> wrapper and the <code>.addMigrations(...)</code> list
<code>entity/</code>	45 entities: animals and identifiers, calf registrations, treatments, movements, mortalities, costs, farmers, locations and pens, feed cribs and entries, users and roles, lookups, and sync bookkeeping
<code>dao/</code>	DAOs, mostly one per file; lookup DAOs share <code>LookupDaos.kt</code>
<code>repository/</code>	Shared repositories: <code>PendingSyncRepository</code> , <code>SyncRunRepository</code> , <code>SyncPolicyEnforcer</code> , <code>SyncWarningPolicy</code> , <code>FarmerRepository</code> , <code>AnimalHistoryRepository</code> and others
<code>security/</code>	KeyStore and passphrase providers, <code>CredentialHasher</code> , <code>PinLockoutManager</code> , <code>TokenProvider</code> and its registries, <code>UnauthorizedReason</code>
<code>util/</code>	<code>FileNamingUtils</code> / <code>BatchNaming</code> (batch names), <code>TagNamingUtils</code> , <code>SyncRunDisplay</code>
<code>*Seed.kt</code>	Lookup seeds: roles, cost types, farmer business roles, rations, other lookups



The non-negotiable rules

Phones can hold unsynced data for days. Losing it is the worst failure this app can have; a real incident is written up in `docs/database/incident-2026-09-r0-migration-data-loss.md`.

NEVER LOSE FIELD DATA

- Never use `fallbackToDestructiveMigration` or any other destructive fallback. Every version step needs an explicit migration that keeps the data.
- Values that cannot be mapped go into quarantine or legacy tables. Never drop them.
- Backfill GUIDs before adding a unique index on them.
- Never delete a local record because a sync failed. Mark it, keep it, retry.
- The only deliberate deletion of unsynced data is the audited Day-7 policy wipe in `SyncSecurityDao`.

Keep DAO calls inside repository classes. ViewModels and UI never touch a DAO.

Changing the schema

Any change to an entity (new table, column, index, type) needs every one of these steps:

- 1 Add or edit the entity in `entity/` and its DAO in `dao/`. Add a new entity to the `entities = [...]` list and a DAO accessor on `BeefTechDatabase`.
- 2 Bump `BeefTechDatabase.VERSION` by one.
- 3 Add `MIGRATION_n_n+1` to the `BeefTechDatabase` companion object. Write the SQL to match exactly what Room generates for the entity (column types, `NOT NULL`, defaults, index names); the exported schema JSON shows you the expected `createSql`.
- 4 Register it in `DatabaseFactory.kt`'s `.addMigrations(...)`, wrapped in `guarded(...)`, which turns any failure into a `MigrationExecutionException`.
- 5 Build once so Room exports the new schema JSON, and commit it under `android/database/schemas/.../`.
- 6 Add a migration test (next section).
- 7 Run `./gradlew :android:database:connectedAndroidTest` on an emulator.

A typical additive migration looks like this:

```
val MIGRATION_47_48 = object : Migration(47, 48) {
    override fun migrate(db: SupportSQLiteDatabase) {
        // Additive and idempotent where possible; existing rows keep their data.
        db.execSQL("ALTER TABLE `feed_crib_entries` ADD COLUMN `note` TEXT")
    }
}
```

VERSION 15

Version 15 is deliberately unused; the chain goes `14→16`. See the comment above `MIGRATION_14_16`.



Testing a migration

Migration tests live in `android/database/src/androidTest/java/com/beeftech/database/` and use Room's `MigrationTestHelper` with the committed schema JSON. Copy `Migration30To31Test.kt` or `Migration16To17Test.kt`. A good migration test:

- creates the database at the old version and inserts realistic rows, including unsynced ones;
- runs the migration and validates the result against the new schema;
- reads the rows back and checks that nothing was lost or changed by accident.

These tests need a real SQLite implementation, so they cannot run as JVM unit tests. CI runs them on an API 30 emulator ([Chapter 9](#)).

Deferred database follow-ups are tracked in `docs/database/future-checks.md`; the schema is drawn in `docs/database/schema-diagram.svg`.



CHAPTER 5

Offline storage and sync

Life of a record

A record moves through a small set of states. Only the server's per-record answer moves it to `SYNCED`.

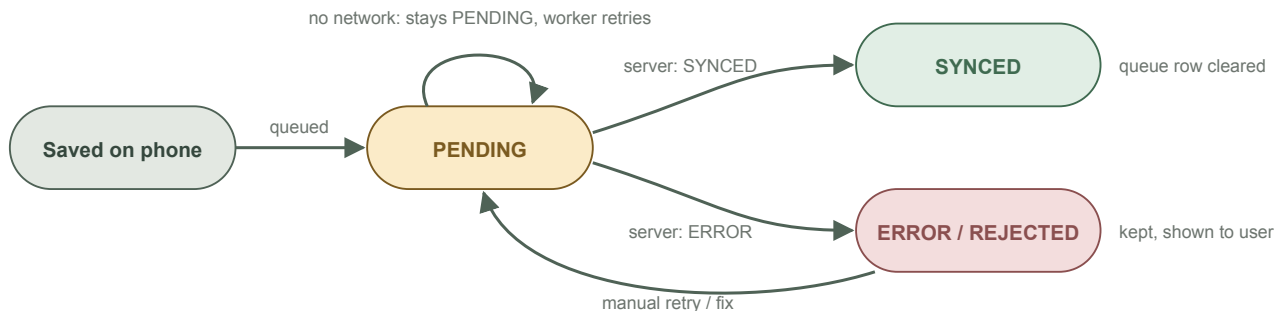


Figure 5.1 — Sync states. A failed request never changes a record; only a per-record server result does.

The pending_sync queue

`PendingSync` (`entity/PendingSync.kt`) is the cross-module queue: one row per record still to send, with `entityType`, `entityId` (the record GUID), `operation`, `payload`, `createdAt`, `retryCount` and the owning `user_id`. Use it through `PendingSyncRepository`:

Call	Use it to
<code>queueOperation(type, id, op, payload)</code>	Queue a record right after saving it
<code>getPendingOperations()</code> / <code>getAllPendingOperations()</code>	Find what is waiting (the first respects the retry limit)
<code>markSyncSuccessful(id)</code> / <code>markEntitySyncSuccessful(...)</code>	Clear the queue row once the server said <code>SYNCED</code>
<code>markSyncFailed(id, ...)</code> / <code>resetRetryCount(...)</code>	Count a failed attempt, or give a manual retry fresh attempts
<code>observePendingCount(userId)</code> / <code>observeOldestPendingAt(userId)</code>	Drive the Home counts and the Day-7 policy

The queue also feeds the 7-day policy and the "waiting" counts on Home, so **every synced record type must queue its records here**, even if the feature also tracks `sync_status` on its own table. Newer features (feed crib) treat their own table as the source of truth and repair the queue whenever the two disagree.

LEGACY ROWS

`user_id` is nullable for rows created before database version 31. Those rows are deliberately not attributed to any user, because guessing could lock or wipe the wrong person's data.



Workers and the dispatcher

Each feature has a `CoroutineWorker` that sends its waiting records. `SyncAllDispatcher.dispatch` (in `:demoapp`) enqueues all of them as unique one-time work with a `CONNECTED` network constraint:

Unique work name	Worker
<code>farmer-registration-sync</code>	<code>FarmerSyncWorker</code>
<code>calf-registration-network-available-sync</code>	<code>CalfRegistrationSyncWorker</code>
<code>treatment_network_available_sync</code>	<code>TreatmentSyncWorker</code>
<code>animal-movement-scheduled-sync</code>	<code>AnimalMovementSyncWorker</code>
<code>mortality-scheduled-sync</code>	<code>MortalitySyncWorker</code>
<code>cost-scheduled-sync</code>	<code>CostSyncWorker</code>
<code>traceability-outbox-scheduled-sync</code>	<code>TraceabilityOutboxWorker</code>
<code>farmer-animal-link-sync</code>	<code>FarmerAnimalLinkSyncWorker</code>
<code>feed-crib-network-available-sync</code>	<code>FeedCribSyncWorker</code>

The dispatcher is triggered by:

- **Sync now** on Home and **Retry Sync** in Traceability, with `ExistingWorkPolicy.REPLACE` (an explicit request restarts a running worker);
- the 05:00 and 18:00 windows (`ScheduledBatchSyncWorker`), with `KEEP` so a worker already syncing is not started twice.

Each request carries a trigger (`SyncRunSummary.TRIGGER_INPUT_KEY`: morning, evening, manual or auto). Workers wrap their work in `SyncRunRepository.trackRun(module, entityType, trigger) { ... }`, which records a row for the sync history screen.

A well-behaved worker:

```
override suspend fun doWork(): Result {
    val database = DatabaseProvider.getDatabase() ?: return Result.retry() // not unlocked yet
    val tokens = TokenProviderRegistry.get() ?: return Result.retry()
    return try {
        syncRuns.trackRun(SyncRunModule.FEED, listOf(ENTITY_TYPE), trigger) {
            repository.syncPending()
            if (stillWaiting()) Result.retry() else Result.success()
        } catch (e: Exception) {
            Result.retry() // never delete a local record because sync failed
        }
    }
}
```

IMMEDIATE SEND

Some features, such as feed crib, also try to send a record straight after saving it when the phone is online. That is only a shortcut; the worker remains the guarantee.



Batch names

Uploads carry an optional batch name in the format `[FarmCode]-[Project]-[YYYYMMDD]-[HHMMSS]-[DeviceID]`, for example `S001-FEED_CRIB-20261010-141503-emulator_5554`.

- The phone builds it with `FileNamingUtils / BatchNaming` in `:android:database`; the server's copy is `common/FileNaming.kt`. **Keep the regex and the project codes in step on both sides.**
- Project codes: `CALF_REG`, `FARMER_REG`, `TREATMENT`, `MOVEMENT`, `MORTALITY`, `COST`, `TRACE_EVENT`, `FEED_CRIB`, `REPORT`.
- The server (`SyncUploadLog.problemWith`) rejects a present name with `400` if the format is wrong, the project does not match the endpoint, the farm code is not the caller's site, or the device is not the signed-in device. A missing name is accepted as a legacy upload.
- Every upload is logged in `sync_upload_log`; a retry with the same name updates its row.

What the server does with a batch

1. Authenticates the caller and reloads their role, site and device from the database.
2. Checks the batch name (`call.acceptBatch(principal, ProjectCode.X, request.batchName)`).
3. Validates each record on its own. A bad record gets `status = "ERROR"` with a message and does not stop the rest of the batch.
4. Upserts each good record by its GUID, so a retry can never create a duplicate. The submitter and site come from the token, never from the request body.
5. Logs the upload (`SyncUploadLog.record`) and answers with one result per record.

The phone then marks each `SYNCED` result as synced (using the server's `serverSyncedAt` when given), clears its queue row, and keeps every `ERROR` record with its message for the user to see.

The 7-day policy

If a user's data stays unsynced, the app escalates. `SyncPolicyEnforcer` evaluates the age of the oldest pending record per user; `SyncPolicyGate` runs it every 60 seconds in the app and `SyncPolicyActivityGuard` applies a lock in every Activity.

Age of oldest unsynced record	What happens	Who can change it
Days 2, 4 and 6 (defaults)	Warnings of increasing severity	Admins, via <code>PUT /api/sync-policy</code> (days 1–6)
Day 7	Pending local operations are wiped and the account is locked, both audited	Nobody. <code>SyncWarningPolicy.WIPE_DAY = 7</code> is a constant the server cannot move
After a lock	The user can only sign in to report the lock	An admin clears it: <code>POST /api/users/{id}/clear-sync-lock</code>

**BE CAREFUL HERE**

This is the one place the app deletes unsynced data on purpose. Any change to `SyncPolicyEnforcer` , `SyncWarningPolicy` or `SyncSecurityDao` needs tests for the boundary days and for users who share a phone.

Device check-in

`DeviceCheckInWorker` runs after each online sign-in, including for a locked account (after a wipe that is the only time a valid token exists). Each step is independent and the job is retried if any fails:

- pulls reference data (`ReferenceDataSync` , using `?ifVersion=` to skip unchanged data);
- pulls the sync-warning policy (`SyncPolicySync`) and the site's feed cribs;
- uploads the phone's sync security events (`SecurityEventSync`);
- lifts the phone's Day-7 lock once an admin has cleared it on the server (`SyncLockRelease`).

It never reads or writes `pending_sync` , so it cannot block or reorder a sync.

CHAPTER 6

Authentication and security

Online sign-in

Users sign in with a username and a 5-digit PIN. `AuthRepository` (in `:android:authentication`) posts to `POST /api/auth/login` with the username, PIN and device ID (and optionally the device model and app version).

Response	Meaning
200	A JWT (HMAC256, 24 h) and the user, including the BCrypt PIN hash the phone caches for offline sign-in
401	Wrong username or PIN
409	The user is bound to a different device
403	This device has been revoked
423	Locked: 5 wrong PINs lock the account on the server for 5 minutes

The first successful sign-in binds the user to that device (`users.device_assigned_id`). A manager can unbind it.

Offline sign-in

With no network, `AuthRepository` checks the PIN against the cached BCrypt hash (`BCrypt.checkpw`). A user must have signed in online once on that phone first. A successful offline sign-in gives **local access only** for up to seven days: `SessionStore.saveOffline` keeps that window separate from the real JWT expiry, and no token is invented for the server.

On the phone, `PinLockoutManager` blocks further attempts for 1 minute after 5 wrong PINs. This is separate from the server's 5-minute lock.

Sessions and tokens on the phone

- `SessionStore` is the interface; `EncryptedSessionStore` stores the JWT, user and expiry in encrypted preferences. It also implements `TokenProvider`.
- `TokenProviderRegistry` and `CurrentUserIdRegistry` (in `:android:database`) let any module get the token or current user without depending on `:android:authentication`.
- API clients call `tokenProvider.token()` and attach it with `bearerAuth(token)`.
- On a `401`, a client calls `tokenProvider.reportUnauthorized(body)`. The session store decides what that means (expired, revoked, deactivated) and ends the session; the user lands on the login screen. Queued records are never touched.



Checking a request on the server

There is no Ktor `authenticate { }` block. Each handler calls one of the helpers in `AuthUtils.kt` as its first line:

Helper	Returns / does
<code>call.requireAuthPrincipal(jwtService)</code>	The caller's <code>AuthPrincipal</code> , or responds <code>401</code> and returns <code>null</code>
<code>call.requireRole(jwtService, Role.ADMIN, Role.MANAGER)</code>	Same, plus <code>403</code> if the role is not listed
<code>call.requireBearerToken(jwtService)</code>	Only the username; same checks

The check runs in a fixed order: decode and verify the JWT; check the live account and device state (inactive user, revoked device, PIN reset after the token was issued → `401`); then reload role, site and device from the user row. So deactivations, role changes and site moves apply on the very next request, even with an older token.

```
post("/api/things/sync") {  
    val principal = call.requireAuthPrincipal(jwtService) ?: return@post  
    ...  
}
```

Record scope

Reads are scoped by role through `RecordScope` (`RecordScope.kt`):

Role	Scope	Sees
Admin	<code>RecordScope.All</code>	Everything
Manager	<code>RecordScope.Site(siteId)</code>	Their site's records. A manager without a site sees nothing
Worker, or unknown role	<code>RecordScope.User(userId)</code>	Only their own records

Use it in every list, single-record read and dashboard count, and pass the table's `voided_at` column so voided records stay hidden:

```
Table.selectAll().where {  
    principal.recordScope().predicate(Table.submittedByUserId, Table.siteId, Table.voidedAt)  
}
```



Before production

DEVELOPMENT DEFAULTS TO REMOVE

- Set `BEEFTECH_JWT_SECRET` . Without it the server falls back to a fixed dev secret and prints a warning.
- Turn off the dev seed (`BEEFTECH_SEED_DEV=false`); the Render blueprint currently turns it on.
- Replace the demo SQLCipher passphrase with the KeyStore-backed key, with a re-key migration for installed phones.
- Remove `usesCleartextTraffic="true"` from release manifests.
- Stop returning `pin_hash` in the login response, or protect what the phone stores, as part of a reviewed offline-login design.



CHAPTER 7

The backend

Code layout

All server code is in `backend/api/src/main/kotlin/com/beeftech/backend/api/`. Each feature is a set of flat files with a common suffix.

File	Responsibility
<code>*Routes.kt</code>	A <code>fun Route.xRoutes(jwtService, service)</code> extension. Authenticates, parses, calls the service, responds with <code>ApiResponse</code>
<code>*Service.kt</code>	Validation and business rules; per-record results for sync
<code>*Repository.kt</code>	Exposed queries inside <code>newSuspendedTransaction(Dispatchers.IO, db = DatabaseFactory.getDatabase())</code>
<code>*Table.kt</code>	Exposed <code>Table</code> objects (the schema)
<code>*Models.kt</code>	<code>@Serializable</code> request and response DTOs
<code>*SchemaMigration.kt</code>	Idempotent changes to tables that already exist

Sub-packages: `auth/` (login, JWT, users, sites, devices, login security, dev seed), `feedCrib/` (package `... api.feedcrib`) and `common/` (`ApiResponse`, `FileNaming`). Shared helpers: `AuthUtils.kt`, `RecordScope.kt`, `SyncUploadLog.kt`, `PdfGenerator.kt`, `ReportCsv.kt`, `CalfPhotoStore.kt`.

Start-up

`Application.kt` holds `main()`, which starts Netty on `$PORT` (default 8081), and `Application.module()`, which:

- 1 resolves the JDBC URL and calls `DatabaseFactory.init(jdbcUrl)` (`Database.kt`): runs the schema migrations, then `SchemaUtils.create(...)` for every table, then seeds reference data and feed crib codes;
- 2 installs `ContentNegotiation` with `Json { ignoreUnknownKeys = true }`, so a newer app sending extra fields keeps working against an older server;
- 3 seeds the dev site, users and cribs if the dev seed is on;
- 4 builds the repositories and services by hand (there is no DI framework) and registers every route group inside `routing { }`.

Tests call the same `module()`, so anything you wire here is exercised by the route tests.

Configuration

There is no `application.conf`. An environment variable wins over the matching `-D` system property.



Environment variable	System property	Default / purpose
PORT	—	8081
BEEFTECH_DB_URL	beeftech.db.url	jdbc:sqlite:./data/beeftech-backend.db
BEEFTECH_SEED_DEV	beeftech.seed.dev	Seed dev site and users when <code>true</code> . If the env var is set at all it wins
BEEFTECH_MEDIA_DIR	beeftech.media.dir	Calf photo folder; defaults to <code>media/</code> next to the database
BEEFTECH_JWT_SECRET	—	JWT signing secret. Always set in production
BEEFTECH_SMTP_PROVIDER	—	custom (default), gmail or outlook
BEEFTECH_SMTP_HOST / _PORT / _SECURITY	—	Host (custom only); port defaults 587/465/25 for starttls / ssl / none
BEEFTECH_SMTP_USERNAME / _PASSWORD	—	Set both or neither
BEEFTECH_SMTP_FROM	—	Sender; defaults to the username
BEEFTECH_SALES_REP_EMAIL	—	Fallback recipient when a site has no sales rep email

Server schema changes

`SchemaUtils.create` only creates tables that are missing; it never alters an existing one. So:

- **A new table:** define the `Table` object and add it to the `SchemaUtils.create(...)` list in `Database.kt`.
- **A new column on an existing table:** add it to the `Table` object *and* write a `*SchemaMigration` object that adds it with `ALTER TABLE` when `PRAGMA table_info` shows it is missing. Call it in `DatabaseFactory.init` *before* `SchemaUtils.create`. Follow `RecordVoidSchemaMigration.kt`: idempotent, and a no-op when the table does not exist yet.
- Add a `*SchemaMigrationTest` that starts from the old shape and checks existing rows survive.

Anatomy of a route

The feed crib sync endpoint shows every convention in a few lines:

```
/* Submitter and site come from the token, never the body. */
post("/api/feed-crib-entries/sync") {
    val principal = call.requireAuthPrincipal(jwtService) ?: return@post
    val request = call.receive<FeedCribEntrySyncRequest>()

    if (!call.acceptBatch(principal, ProjectCode.FEED_CRIB, request.batchName)) return@post

    val response = feedCribService.sync(request, principal.userId, principal.siteId)
    SyncUploadLog.record(principal, ProjectCode.FEED_CRIB, request.batchName,
        response.results.map { it.status })

    call.respond(ApiResponse(success = true, message = "Feed crib entries synchronized.", data
    = response))
}
```



Every response body is an `ApiResponse<T>(success, message, data)`, including errors.

Voiding, audit, email, reports

- **Voiding** is the only way to correct a synced record; there are no edits. `POST /api/records/{type}/{id}/void` sets `voided_at`, `voided_by_user_id` and `void_reason` and writes the audit log. A new record type that managers should be able to void needs those columns and an entry in `VoidRepository`.
- **Audit log** (`AuditLogTable`): written with `insertAuditRow(...)` by user, site, device, login security, reference data, sync policy, void and restore changes; read through `AuditService`.
- **Email** (`FarmerSalesNotification.kt`): the first sync of a farmer, and of a calf registration, emails a JSON receipt to the site's sales rep (or `BEEFTECH_SALES_REP_EMAIL`). A farmer is emailed at most once (claimed via `farmers.sales_notified_at`). With incomplete SMTP settings the email is logged instead and retried on a later sync.
- **Reports** (`ReportService`, `ReportCsv`, `PdfGenerator` with `PDFBox`): JSON, CSV or PDF via `?format=`. Calf birth certificates are PDFs too.
- **Photos** (`CalfPhotoStore`): raw `image/jpeg`, max 5 MB, stored on disk under the media folder.



CHAPTER 8

API reference

Conventions

Base URL `https://beeftech-backend.onrender.com` (hosted) or `http://<host>:8081` (local). Every route except the public ones needs `Authorization: Bearer <jwt>`.

Role: *any* — any signed-in user; *mgr* — manager or admin (managers limited to their own site); *admin* — admin only (403 otherwise). Responses are `ApiResponse` JSON unless stated.

NOTE

`backend/api/requests.http` is out of date (it sends `password` and calls the removed `/api/auth/register`). Use the `curl` examples in Chapter 2 or the route tests instead.

Public

Method	Path	Purpose
GET	<code>/</code> · <code>/health</code> · <code>/api/farm-traceability</code>	Liveness text (Render uses <code>/health</code>)
POST	<code>/api/auth/login</code>	<code>{username, pin, device_id, device_model?, app_version?}</code> → JWT and user

Field data

Method	Path	Role	Purpose
GET	<code>/api/profile</code>	any	Current username
POST	<code>/api/calf-registrations/sync</code>	any	Batch upsert by GUID
GET	<code>/api/calf-registrations</code> · <code>/api/calf-registrations/{tagNumber}</code>	any	List (scoped) / single
GET	<code>/api/calf-registrations/assignment-candidates</code>	mgr	Calves on the manager's site assignable to <code>?farmerId=</code>
PUT · GET	<code>/api/calf-registrations/{tag}/photo</code>	any	Upload (raw JPEG, ≤ 5 MB; 413 / 415) / download
GET	<code>/api/calf-registrations/{tag}/certificate</code>	any	Birth certificate PDF
POST	<code>/api/farmers/sync</code>	any	Batch upsert; sends the sales email once per farmer
GET	<code>/api/farmers</code> · <code>/api/farmers/{farmerId}</code>	any	List / single



Method	Path	Role	Purpose
POST	/api/treatments/sync	any	Batch upsert
GET	/api/treatments · /{animalId} · /reference-data	any	List / per animal / legacy reference data
POST	/api/animal-movements/sync	any	Batch upsert
POST	/api/mortalities/sync	any	Batch upsert
POST	/api/costs/sync	any	Batch upsert
GET	/api/{animal-movements mortalities costs} · /{animalId}	any	List / per animal
POST	/api/farmer-animal-links/sync	any	Upload one farmer–animal assignment, checked against site and saved link
POST	/api/farmer-animal-links/diagnose	any	Why pending assignments (≤ 50) are not syncing
POST	/api/traceability-events/sync	any	Generic outbox (<code>entityType</code> , <code>recordGuid</code> , <code>payload</code>), upsert by GUID
GET	/api/feed-cribs	any	Site's cribs, codes and last <code>days</code> (default 3, max 14) of entries; admins pass <code>?siteId=</code>
POST	/api/feed-crib-entries/sync	any	Batch upsert by <code>recordguid</code> ; also updates <code>feed_cribs.current_ad</code>

Reference data and sync policy

Method	Path	Role	Purpose
GET	/api/reference-data	any	Diseases, treatment types, cost types; <code>?ifVersion=N</code> → <code>unchanged</code>
POST · PATCH	/api/reference-data/{kind} · /{kind}/{id}	admin	Add a value / activate or deactivate
GET · PUT	/api/sync-policy	any · admin	Read / change warning days (1–6) and stale-sync hours (12–336)
POST	/api/sync-security-events	any	Device uploads its security events (≤ 200)
GET	/api/sync-security-events · /locked	admin	Event log / locked users
POST	/api/users/{id}/clear-sync-lock	admin	Clear a Day-7 lock (audited)



Administration

Method	Path	Role	Purpose
GET · POST	/api/users	mgr	List / create (managers: workers on their site)
PATCH	/api/users/{id}	mgr	Update (only admins change role or site)
POST	/api/users/{id}/reset-pin · /unlock-login · /unbind-device	mgr	Reset PIN (also unlocks) / lift lockout / release device
GET	/api/sites	mgr	List sites (managers: their own)
POST · PATCH	/api/sites · /{id}	admin	Create / update (name, 4-char farm code, salesRepEmail)
GET	/api/devices	mgr	List devices
POST	/api/devices/{id}/revoke · /reinstate	admin	Revoke / reinstate with {reason}
GET	/api/login-events · /api/login-security/lockouts	admin	Login history / current lockouts
POST	/api/farmer-animal-links/restore-missing-parent	mgr	Recreate a missing farmer or calf an assignment needs; verificationReason 15–500 chars
POST	/api/records/{type}/{id}/assign-site	admin	Give a legacy record without a site a site
POST	/api/records/{type}/{id}/void	mgr	Void with {reason} (≤ 500 chars)
GET	/api/records/{type}	mgr	Review list, including voided
GET	/api/audit-log	mgr	Audit log with filters
GET	/api/dashboard/summary	mgr	Dashboard figures (?siteId)
GET	/api/reports/{report}	mgr	mortality , treatment-cost , cost-per-animal , calf-registrations , worker-productivity ; ?format=json csv pdf

Void and review {type} values: calf-registrations , treatments , farmers , animal-movements , mortalities .



CHAPTER 9

Testing and CI

Kinds of test

Command	Runs	Device?
<code>./gradlew testDebugUnitTest test</code>	Every JVM unit test (exactly what CI runs)	No
<code>./gradlew :backend:api:test</code>	Ktor route, auth, admin, email and schema-migration tests	No
<code>./gradlew :android</code> <code>:<module>:testDebugUnitTest</code>	One module's unit tests	No
<code>./gradlew lintDebug</code>	Android Lint in every module	No
<code>./gradlew</code> <code>:android:database:connectedAndroidTest</code>	SQLCipher, keystore, DAO, sync and migration tests	Yes
<code>./gradlew :android:tag-scanner:connectedAndroidTest</code>	Ear-tag recognition over sample images	Yes

Run a single class with `--tests`:

```
./gradlew :android:feed-crib:testDebugUnitTest --tests
"com.beeftech.feedcrib.SlotAllocatorTest"
./gradlew :backend:api:test --tests "com.beeftech.backend.api.VoidRoutesTest"
```

Android unit tests

Feature logic is tested on the JVM with hand-written fakes rather than a mocking library. `android/feed-crib/src/test/` is the model:

- `fakes/FakeFeedCribDao.kt` and `FakePendingSyncDao.kt` — in-memory DAOs;
- `fakes/FakeFeedCribServer.kt` — a scriptable backend behind Ktor's `MockEngine`, with an `offline` switch;
- `fakes/FakeTokenProvider.kt` — a token source;
- `FeedCribRepositoryTest`, `FeedCribViewModelTest`, `FeedCribApiClientTest`, `SlotAllocatorTest`.

Because repositories take their clock, time zone and user ID as constructor parameters, tests can pin the time of day and the signed-in user. Modules that log through `android.util.Log` set `unitTests.isReturnDefaultValues = true` so logging does not throw on the JVM.

Test what matters for field data: a save survives a failed sync; a retry does not duplicate; an `ERROR` result is kept and shown; the queue and the record table agree afterwards.



Backend route tests

Backend tests start the real `module()` in Ktor's `testApplication` against a fresh temporary SQLite file, with the dev seed on. Helpers live in `AdminTestHelpers.kt`:

```
@Test
fun `a worker sees only their own records`() = testApplication {
    startAdminApp() // temp DB + dev seed + module()
    val client = createClient { }
    val worker = client.adminLogin("jvdm", "30003") // returns a JWT
    val response = client.adminSend("GET", "/api/treatments", worker)
    assertEquals(HttpStatusCode.OK, response.status)
}
```

Pass the same file to `startAdminApp(file)` twice to simulate a restart on the same database (useful for schema migrations).

FLAKY SUITE

`:backend:api:test` is known to be flaky: a different single test can fail on each run, probably because tests share static state (N5 in `docs/database/future-checks.md`). Re-run before assuming a regression, and do not add new shared mutable state.

`FarmerSalesNotificationLiveTest` sends a real email and is skipped unless `BEEFTECH_SMTP_LIVE_TEST=true`.

Continuous integration

`.github/workflows/ci.yml` runs on every push and pull request to `main`, on JDK 17:

Job	Command	Reports artifact
Unit tests	<code>./gradlew testDebugUnitTest test --continue</code>	<code>test-reports</code>
Android Lint	<code>./gradlew lintDebug --continue</code>	<code>lint-reports</code>
Database migration tests (emulator)	<code>:android:database:connectedAndroidTest</code> on API 30, x86_64, KVM	<code>database-instrumented-test-reports</code>

None of the jobs is a required check yet; do not merge on red anyway.

Contributing

- Branch from `main`: `feature/<short-description>`, `fix/...`, `docs/...`.
- Commit messages follow Conventional Commits with the module as scope, e.g. `feat(feed-crib): add crib note`, `fix(backend): ...`, `docs(user-manual): ...`.
- Before opening a pull request run `./gradlew build`, `./gradlew testDebugUnitTest test` and `./gradlew lintDebug`. Run `connectedAndroidTest` for any change under `:android:database`.
- Update the README, this manual and, for user-visible changes, the User Manual in the same pull request.



Recipes

Adding a new synced record type

The most common large change. Feed crib (schema 47) went through every step below; read its files next to this list.

On the server

- 1 **Table.** Create `XTable.kt` with a unique `record_guid`, the record's fields, `submitted_by_user_id`, `site_id`, `device_id`, capture and server-sync timestamps, and `voided_at` / `voided_by_user_id` / `void_reason` if it can be voided. Add it to `SchemaUtils.create(...)` in `Database.kt`.
e.g. `feedCrib/FeedCribTable.kt`
- 2 **Models.** `@Serializable` sync request (`deviceId`, `batchName?`, `records`), per-record result (`recordGuid`, `status`, `message?`, `serverSyncedAt?`) and response.
- 3 **Repository.** `upsert` by GUID inside `newSuspendedTransaction`; scoped list and single reads using `RecordScope.predicate(...)`.
- 4 **Service.** Validate each record; return `SYNCED` or `ERROR` per record, never throw for the batch.
- 5 **Project code.** Add a value to `FileNaming.ProjectCode` (server) *and* the phone's copy in `:android:database`.
- 6 **Routes.** `POST /api/x/sync` following [the anatomy above](#), plus `GET` reads. Register them in `Application.module()`.
- 7 **Tests.** A `XRoutesTest` covering: unauthenticated → 401, upsert is idempotent, submitter/site come from the token, one bad record does not fail the batch, scope per role, bad batch name → 400.

On the phone

- 1 **Entity and DAO** in `:android:database`, with `record_guid` as the key, `sync_status`, `sync_error`, `synced_at`, `user_id`, `device_id`, `captured_at`. Index `sync_status`.
- 2 **Migration:** the full checklist in [Chapter 4](#), including the schema JSON and a migration test.
- 3 **API client** in the feature's `data/`: `baseUrl = BackendConfig.baseUrl`, `bearerAuth(token)`, report 401, return `Result<T>`. Send a batch name.
- 4 **Repository:** save locally → `queueOperation(ENTITY_TYPE, guid, "UPSERT", ...)` → try `syncPending()`. Apply results per record; clear the queue row only on `SYNCED`.
- 5 **Worker:** the shape in [Chapter 5](#), wrapped in `SyncRunRepository.trackRun`. Add a `SyncRunModule` value if the feature is new.
- 6 **Dispatcher:** add a unique work name and the worker to `SyncAllDispatcher`, and update its test in `demoapp/src/test/`.
- 7 **Home and policy:** make sure the pending count on Home and the 7-day policy see the new entity type (they read `pending_sync`).
- 8 **UI:** ViewModel + screens; a tab in `AppBar` or an entry from an existing screen.
- 9 **Tests:** repository and ViewModel tests with fakes ([Chapter 9](#)).



10 Docs: README module table and API reference, this manual, the User Manual.

Adding an endpoint

- 1 Add the DTOs to the feature's `*Models.kt`.
- 2 Add the query to the repository and the rule to the service.
- 3 Add the route to the feature's `*Routes.kt`. First line: `requireAuthPrincipal` or `requireRole`. Use `RecordScope` for reads. Respond with `ApiResponse`.
- 4 If it is a new route group, register it in `Application.module()`.
- 5 Write route tests for each role (allowed and refused).
- 6 Add a method to the Android API client that will call it, and add the endpoint to the README and [Chapter 8](#).

Adding a manager or admin screen

Management screens in `:android:management` are online-only: they call the server through `ManagementApiClient` and show a "needs connection" notice when offline. They do not write to Room or `pending_sync`.

- 1 Add the call to `ManagementApiClient` (`management/data/`).
- 2 Add a ViewModel in `management/viewmodel/` exposing loading, content, error and offline states.
- 3 Add the screen in `management/ui/`. Admin-only pages go inside the Admin area; manager pages get an `AppBar` value and an entry in `moreTabsFor`.
- 4 Protect the endpoint on the server with `requireRole`; never rely on the screen being hidden.
- 5 Add ViewModel tests in `android/management/src/test/`.

Adding a dependency

- Add it to `gradle/libs.versions.toml` and reference it as `libs....`. Older modules still hard-code coordinates; new code should use the catalog.
- Repositories can only be declared in `settings.gradle.kts` (`FAIL_ON_PROJECT_REPOS`).
- Match the Ktor client version (3.0.3) and Compose BOM already used by the module you are editing.
- If the dependency brings SQLCipher or BouncyCastle into a new module, copy the `packaging { resources { excludes += ... } }` block from `:android:database` to avoid duplicate `META-INF` errors.



CHAPTER 11

Building, releasing and deploying

Building the APK

```
./gradlew :demoapp:assembleDebug # → demoapp/build/outputs/apk/debug/  
./gradlew :demoapp:assembleRelease # talks to https://beeftech-backend.onrender.com/
```

- `applicationId` is `com.beeftech.demoapp`, `versionName` `1.0`; bump `versionCode` and `versionName` in `demoapp/build.gradle.kts` for each release so phones accept the update.
- No release signing config is committed. Sign release APKs with the team's keystore outside the repository; never commit it.
- On phones the app is still labelled **demoapp** in the app drawer.
- An update must open the existing encrypted database and migrate it. Test an upgrade over the previous release with unsynced data on the phone before shipping.

Deploying the backend

The `Dockerfile` builds `:backend:api:installDist` on Temurin 17 and runs `/app/bin/api` on a JRE 17 image, exposing 8081. `render.yaml` deploys it to Render as `beeftech-backend`:

- auto-deploys on every commit to the tracked branch; health check `/health`;
- generates `BEEFTECH_JWT_SECRET`; SMTP goes through the Brevo relay; `BEEFTECH_SMTP_USERNAME`, `_PASSWORD`, `_FROM` and `BEEFTECH_SALES_REP_EMAIL` are entered in the Render dashboard (`sync: false`) and never committed;
- `BEEFTECH_SEED_DEV=true`, so the dev users exist on the hosted server.

NO PERSISTENT DISK

The Render service has no disk, so the hosted SQLite database and calf photos reset on every redeploy or restart. Treat it as a demo environment. Production needs a persistent volume (and `BEEFTECH_DB_URL` / `BEEFTECH_MEDIA_DIR` pointing at it) or a managed database.

To try the image locally:

```
docker build -t beeftech-backend .  
docker run -p 8081:8081 -e BEEFTECH_SEED_DEV=true -e BEEFTECH_JWT_SECRET=dev-only beeftech-backend
```

Rebuilding the manuals

Both manuals are HTML rendered to PDF by headless Chromium. They share one stylesheet and logo in `docs/user-manual/`:



```
docs/user-manual/      manual.html · cover.html · manual.css · assets/ · screenshots/ ·  
build.sh  
docs/developer-manual/ manual.html · cover.html · developer.css · build.sh
```

```
docs/user-manual/build.sh      # → BeefTech-User-Manual.pdf  
docs/developer-manual/build.sh # → BeefTech-Developer-Manual.pdf
```

They need `chromium` (or `google-chrome`; set `CHROME=` to override) and `pdffunite` from poppler-utils. Edit the HTML, rebuild, and commit the HTML and the PDF together.



CHAPTER 12

Troubleshooting and reference

Troubleshooting

Symptom	Fix
Unable to download toolchain matching ... languageVersion=17	Install a JDK 17, or point Gradle at one: <code>- Porg.gradle.java.installations.paths=/path/to/jdk-17</code>
SDK location not found	Create <code>local.properties</code> with <code>sdk.dir=...</code>
Plugin [id: 'com.android.application' ...] was not found	Gradle cannot reach Google Maven / Maven Central. Check proxy or VPN
Duplicate META-INF resources	Copy the <code>packaging</code> excludes from <code>:android:database</code>
Room schema export fails with <code>AbstractMethodError</code>	KSP2 must stay on: <code>ksp.useKSP2=true</code> in <code>gradle.properties</code>
Lint crashes reading Kotlin metadata of a fake DAO	Set <code>lint { ignoreTestSources = true }</code> in that module, as <code>feed-crib</code> does
App cannot reach the backend	Emulator: backend running on 8081? Physical phone: reinstall with <code>- Pbeeftech.baseUrl=http://<lan-ip>:8081/</code>
Sign-in returns 409	User bound to another device; unbind it or reset the local backend database
Worker logs "Database is not initialized"	Expected before the first unlock after a process start; it retries
Room cannot verify the data integrity / migration missing	You changed an entity without bumping <code>VERSION</code> and adding a migration (Chapter 4). Never "fix" this by uninstalling on a field phone
A backend test fails once, passes on re-run	Known flakiness (Chapter 9)
Gradle behaves strangely after a branch switch	<code>./gradlew --stop && ./gradlew clean</code>

Known gaps

- The demo SQLCipher passphrase, the JWT fallback secret and the dev users are hard-coded for development.
- `usesCleartextTraffic=true` is set in main manifests, so release builds allow HTTP too.
- The login response includes the user's `pin_hash` (used for offline sign-in).
- The Render deployment has no persistent disk and seeds the dev users.
- `CalfRegistrationViewModel` and `TreatmentViewModel` re-enqueue the legacy 15-minute periodic jobs that `ScheduledSyncScheduler` cancels.



- The biometric prompt code in `:android:authentication` is unused.
- `backend/authentication`, `backend/sync`, `docs/architecture/` and `docs/testing/` are empty.
- `backend/api/requests.http` is stale; stale standalone Gradle files remain under `android/`.
- Dependency declarations are mixed between the version catalog and hard-coded coordinates; Compose BOM versions differ between modules.
- CI jobs are not required checks; `:backend:api:test` is flaky.

Toolchain and versions

Component	Version	Component	Version
Gradle wrapper	8.13	Ktor client / server	3.0.3
Daemon JVM / JVM target	17	Exposed	0.56.0
Android Gradle Plugin	8.13.2	SQLite JDBC	3.46.1.3
Kotlin	2.0.21	PDFBox	2.0.30
KSP	2.0.21-1.0.28 (KSP2)	Jakarta Mail (Angus)	2.0.3
Room	2.8.4	java-jwt / jBCrypt	4.4.0 / 0.4
SQLCipher for Android	4.17.0	ML Kit text recognition	16.0.1
compileSdk	35 / 34 (per module)	minSdk	24 (23 for database)

Other documents

Document	What it covers
<code>README.md</code>	Quick start and the same reference material in Markdown
<code>docs/user-manual/BeefTech-User-Manual.pdf</code>	The app from the user's side, with screenshots
<code>docs/database/incident-2026-09-r0-migration-data-loss.md</code>	Why the migration rules exist
<code>docs/database/future-checks.md</code>	Deferred database and backend follow-ups
<code>docs/database/schema-diagram.md</code> / <code>.svg</code>	The local schema
<code>docs/database/referential-integrity-audit-2026-09-28.*</code>	Integrity audit and its SQL
<code>docs/requirements/</code>	Client feedback gaps; farmer sales email plan
<code>docs/design/field-ready-redesign-plan.md</code>	The field-ready UI redesign
<code>docs/farmer-calf-linking-proposal.md</code>	Farmer-calf linking proposal



Glossary

ADI	Average daily intake (kg) for a feed crib
Batch name	<code>[FarmCode]</code> - <code>[Project]</code> - <code>[YYYYMMDD]</code> - <code>[HHMMSS]</code> - <code>[DeviceID]</code> label sent with each upload
Check-in	<code>DeviceCheckInWorker</code> : reference data and policy down, security events up
Day-7 wipe / lock	The audited deletion of a user's pending operations and account lock after 7 days unsynced
Device binding	A user may only sign in on the device they first used, until unbound
Farm code	A site's 4-character code (A–Z, 0–9), e.g. <code>S001</code>
GUID / record GUID	The ID a phone gives each record; the server upserts by it
Outbox	The generic <code>/api/traceability-events/sync</code> channel for traceability record types
pending_sync	The cross-module queue of records still to upload
Principal	<code>AuthPrincipal</code> : the server's view of the caller (user, role, site, device)
Record scope	Which records a caller may read: all, their site, or their own
Site	A feedlot or farm; managers and workers belong to one
Slot	Feed crib reading block: <code>MORNING</code> , <code>MIDDAY</code> or <code>EVENING</code>
Void	Hiding a synced record with a reason; the only correction mechanism